

NETOPS MCP ASSISTANT

Safe Linux Network Automation

Team T018

CS331 Computer Networks Final Project

Project Description

This project builds an **intelligent assistant using MCP** that helps network administrators configure devices securely and efficiently. It interprets user commands and applies configurations like firewall rules and bandwidth limits based on predefined policies and network context.

Expected Outcomes

- Apply network configurations from user commands
- Validate changes through monitoring tools
- Provide structured, rule-based automation for secure and optimized settings

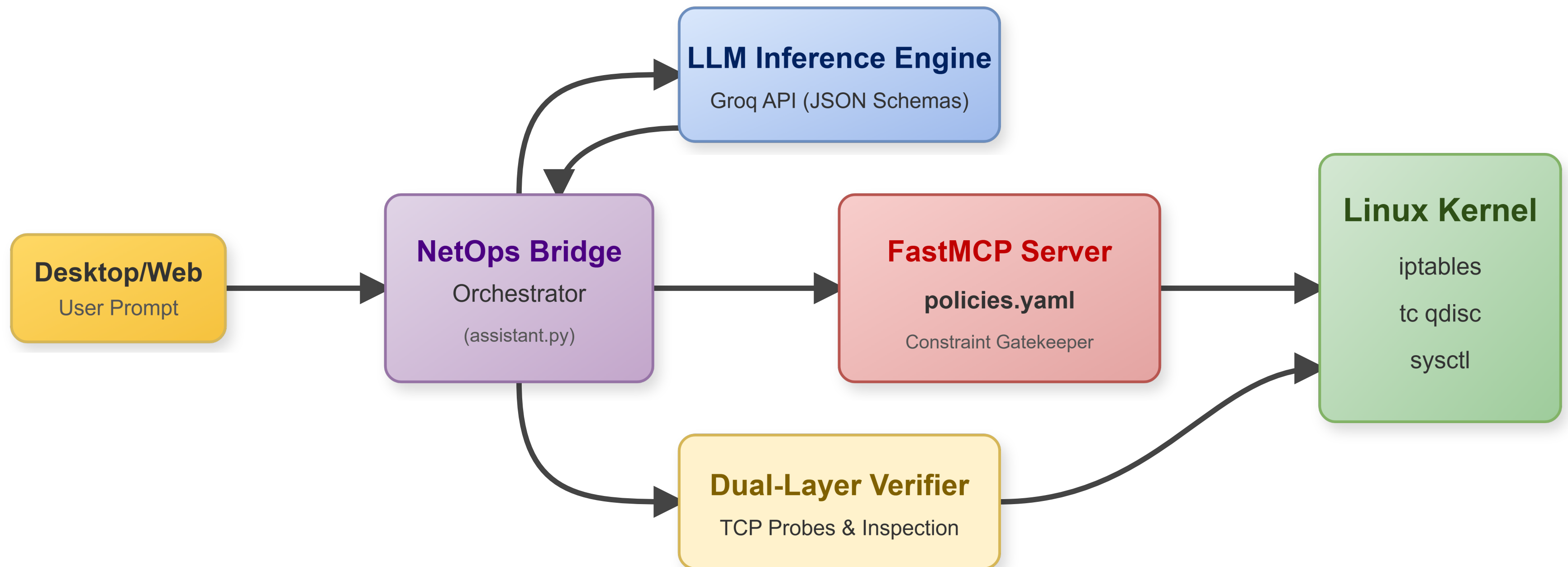
Deliverables

Working assistant with example commands, rule files, deployment scripts, test cases, and a final report explaining setup, use cases, and validation results.

Tools/Technologies

MCP frameworks (FastMCP, Anthropic MCP SDK), network tools (iptables, tc), scripting (Python, Bash), configuration files (YAML/JSON), monitoring tools (ping, iperf3), Docker for deployment.

System Architecture Workflow



UI & Optimizations

Friendly User Interface

- Desktop GUI (pywebview) and Web Browser (Flask) forms available.
- Live subsystem health cards and real-time benchmark visualization.

Network Profiles & Fallback State

- **Gaming:** Uses bbr congestion control and fq_code1 queuing.
- **Streaming:** Auto-tunes receive windows (tcp_rmem) and fq pacing.
- **Broadcasting:** Tunes send windows (tcp_wmem) and ephemeral ports.
- **Fallback State:** Safe checkpointing (checkpoint.json) restores defaults instantly on errors.

Simple LLM with API

Added a simple, low-latency LLM via API (Groq LPU). Processes natural language directly to structured JSON tools, avoiding local overhead.

Non-Functional Testing Parameters

Performance:

Delivers sub-second intent reasoning using Groq API. The FastMCP Server uses `stdio` for zero-overhead IPC latency. Subprocess arrays execute without a shell for fast dispatch.

Scalability:

Stateless FastMCP architecture efficiently handles concurrent tool requests. The codebase is separated into independent operational modules (e.g., `net_ops.py`, `benchmark.py`) enabling easy expansion.

Reliability & Fallbacks:

Dual-Layer Verifier tests real TCP sockets alongside kernel table states. Before any `sysctl` mutations, `checkpoint.json` takes a snapshot, ensuring instant rollback (zero downtime) if settings fail.

Security & Safety:

Declarative `policies.yaml` safeguards ports 22, 53, 5000. Argument sanitization and total avoidance of `shell=True` prevent command injection.

Challenges Faced

- **LLM connection to MCP & Architecture Changes:**

Migrating from basic shell execution to the strict boundaries of the Model Context Protocol required careful schema design. We had to enforce the rule: "The LLM is an interpreter, not a security authority."

- **Rebuilding Validation & Verification:**

Traditional `ping` wasn't enough (it only tests ICMP). We had to rebuild validation with a Dual-Layer Verifier that performs active TCP probing and kernel table inspection.

- **Docker & Privilege Boundaries:**

Running iptables and tc inside Docker required granting NET_ADMIN and NET_RAW capabilities to avoid permission denials without fully compromising container security.

Results

```
root@d7b4b10d83e5:/app# iptables -L INPUT -v -n --line-numbers
Chain INPUT (policy ACCEPT 457K packets, 19G bytes)
num  pkts bytes target    prot opt in     out     source    destination    tcp dpt:9999
1      2   120 DROP      tcp  --  *      *       0.0.0.0/0  0.0.0.0/0
2      2   120 DROP      tcp  --  *      *       0.0.0.0/0  0.0.0.0/0      tcp dpt:1111
root@d7b4b10d83e5:/app# |
```

NETWORK OPERATION EXIT 0

```
$ iptables -L INPUT -v -n --line-numbers
```

Chain INPUT (policy ACCEPT 457K packets, 19G bytes)

num	pkts	bytes	target	prot	opt	in	out	source	destination	
1	2	120	DROP	tcp	--	*	*	0.0.0.0/0	0.0.0.0/0	tcp dpt:9999
2	2	120	DROP	tcp	--	*	*	0.0.0.0/0	0.0.0.0/0	tcp dpt:1111

ACTIVE FIREWALL RULES

Chain INPUT (policy ACCEPT 457K packets, 19G bytes)										
num	pkts	bytes	target	prot	opt	in	out	source	destination	
1	2	120	DROP	tcp	--	*	*	0.0.0.0/0	0.0.0.0/0	tcp dpt:9999
2	2	120	DROP	tcp	--	*	*	0.0.0.0/0	0.0.0.0/0	tcp dpt:1111

⚡ Network Benchmark: Before vs After				GAMING BENCHMARK
METRIC	BEFORE	AFTER	CHANGE	
Avg Ping Latency	23.585 ms	20.72 ms	▼12.1% better	
Ping Jitter (mdev)	5.274 ms	5.182 ms	✓ Stable (< 2.5 ms)	
Packet Loss	0 %	0 %	✓ 0% Loss	
TCP Connect Time	19.12 ms	15.73 ms	▼17.7% better	

MEASURED BENCHMARK: GAMING PROFILE

Optimization Goal: Minimize latency (ping) and jitter